

HAProxy Load Balancing for PostgreSQL

Table of Contents

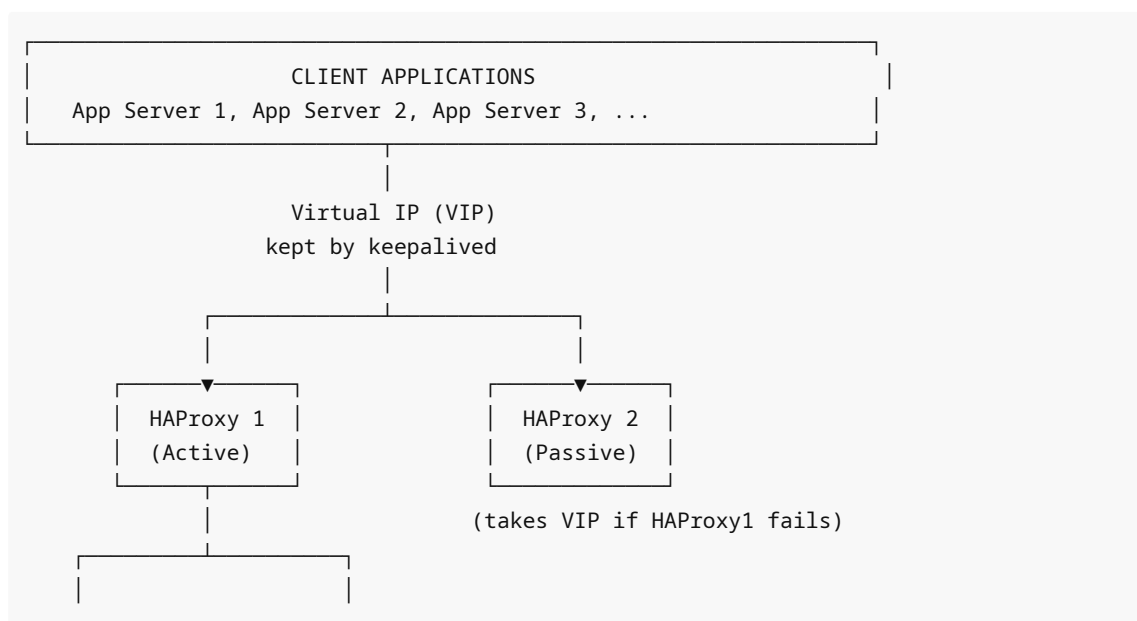
1. [Learning Objectives](#)
 2. [HAProxy Architecture for PostgreSQL](#)
 3. [Installation](#)
 4. [Complete HAProxy Configuration](#)
 5. [Health Checks via Patroni API](#)
 6. [Routing Reads to Replicas](#)
 7. [HAProxy Stats Dashboard](#)
 8. [High Availability for HAProxy](#)
 9. [Integration Patterns](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Performance Considerations](#)
 13. [Interview Questions](#)
 14. [Exercises and Solutions](#)
-

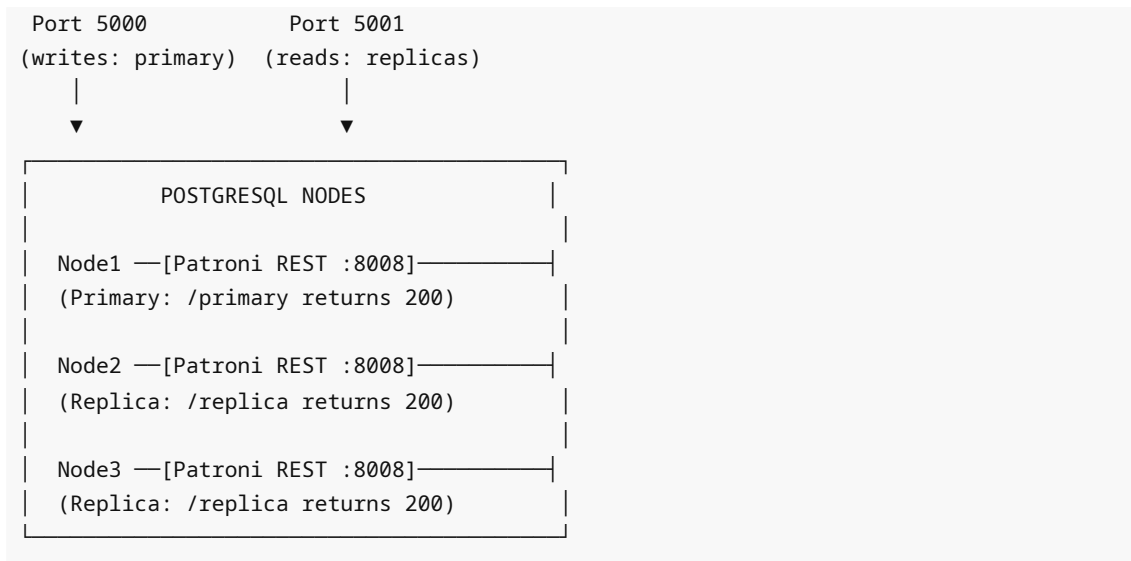
Learning Objectives

By the end of this module, you will be able to:

- Configure HAProxy to route read/write traffic to the correct PostgreSQL nodes
 - Use Patroni REST API endpoints for HAProxy health checks
 - Set up read replica load balancing
 - Make HAProxy itself highly available with Keepalived
 - Monitor connection distribution and backend health
 - Implement graceful failover routing
-

HAProxy Architecture for PostgreSQL





HAProxy uses HTTP health checks against Patroni's REST API to determine which backend is the current primary and which are replicas.

Installation

```
# Ubuntu/Debian
sudo apt-get install -y haproxy

# RHEL/CentOS
sudo yum install -y haproxy

# Verify
haproxy -v

# Enable and start
sudo systemctl enable haproxy
sudo systemctl start haproxy
```

Complete HAProxy Configuration

```
# /etc/haproxy/haproxy.cfg
# Production HAProxy configuration for PostgreSQL with Patroni

#-----
# GLOBAL SETTINGS
#-----

global
    maxconn 10000                # Maximum total connections
    log /dev/log local0 info     # Syslog logging
    log /dev/log local1 notice
    chroot /var/lib/haproxy
```

```
stats socket /run/haproxy/admin.sock mode 660 level admin expose-fd listeners
stats timeout 30s
user haproxy
group haproxy
daemon

# SSL/TLS settings
ssl-default-bind-options ssl-min-ver TLSv1.2
ssl-default-bind-ciphers ECDH+AESGCM:DH+AESGCM:ECDH+AES256:DH+AES256

#-----
# DEFAULTS
#-----
defaults
    log global
    mode tcp                # TCP mode for PostgreSQL (not HTTP)
    option tcplog           # TCP logging
    option dontlognull      # Don't log null connections (health checks)
    timeout connect 5s      # Time to establish connection to backend
    timeout client 30m      # Idle client connection timeout
    timeout server 30m      # Idle server connection timeout
    timeout check 5s        # Health check timeout
    retries 3               # Retry failed connections

#-----
# STATS DASHBOARD
#-----
frontend stats
    mode http
    bind *:7000
    stats enable
    stats uri /stats
    stats realm HAProxy\ Statistics
    stats auth admin:haproxy_admin_password
    stats refresh 30s
    stats show-legends
    stats show-node
    option httplog

#-----
# PRIMARY (READ-WRITE) PORT
#-----
frontend postgres_primary_frontend
    bind *:5000
    mode tcp
    default_backend postgres_primary

backend postgres_primary
    mode tcp
    balance roundrobin      # Only 1 server will be active (the primary)
                           # Others fail health checks
```

```

# TCP keepalive settings
option tcp-check
timeout connect 5s
timeout check 5s

# Health check: Patroni /primary endpoint
# Returns HTTP 200 only if this node is the current primary
option httpchk GET /primary HTTP/1.0
http-check expect status 200

# All three PostgreSQL nodes
# Only the one that is currently primary will pass health check
server pg-node1 192.168.1.100:5432 check port 8008 inter 2000 fall 3 rise 2
server pg-node2 192.168.1.101:5432 check port 8008 inter 2000 fall 3 rise 2
server pg-node3 192.168.1.102:5432 check port 8008 inter 2000 fall 3 rise 2

# Server options explained:
# check          : Enable health checking
# port 8008       : Check port (Patroni REST API), not PostgreSQL port
# inter 2000      : Check every 2 seconds
# fall 3          : Mark DOWN after 3 consecutive failures
# rise 2          : Mark UP after 2 consecutive successes (fast recovery)

#-----
# READ REPLICAS PORT
#-----
frontend postgres_replica_frontend
    bind *:5001
    mode tcp
    default_backend postgres_replicas

backend postgres_replicas
    mode tcp
    balance roundrobin          # Load balance across available replicas
    option tcp-check

# Health check: Patroni /replica endpoint
# Returns 200 only if this node is a healthy replica
option httpchk GET /replica HTTP/1.0
http-check expect status 200

# All nodes listed – healthy replicas will accept connections
server pg-node1 192.168.1.100:5432 check port 8008 inter 2000 fall 3 rise 2
server pg-node2 192.168.1.101:5432 check port 8008 inter 2000 fall 3 rise 2
server pg-node3 192.168.1.102:5432 check port 8008 inter 2000 fall 3 rise 2

#-----
# READ REPLICAS WITH PRIMARY FALLBACK
#-----
# Use this if you want reads to go to replicas but fall back to primary
# when no replicas are healthy

```

```

backend postgres_reads_with_fallback
    mode tcp
    balance roundrobin
    option tcp-check
    option httpchk GET /read-only HTTP/1.0    # /read-only returns 200 for primary
too
    http-check expect status 200

    server pg-node1 192.168.1.100:5432 check port 8008 inter 2000 fall 3 rise 2
    server pg-node2 192.168.1.101:5432 check port 8008 inter 2000 fall 3 rise 2
weight 3
    server pg-node3 192.168.1.102:5432 check port 8008 inter 2000 fall 3 rise 2
weight 3
    # Replicas have higher weight (3x more traffic than primary)

#-----
# PGBOUNCER-FRONTED BACKEND
#-----
# If PgBouncer is deployed per-node, route through it

frontend pgbouncer_primary
    bind *:5010
    mode tcp
    default_backend pgbouncer_primary_backend

backend pgbouncer_primary_backend
    mode tcp
    balance roundrobin
    option httpchk GET /primary HTTP/1.0
    http-check expect status 200

    # PgBouncer on each node (port 6432)
    # Health check still against Patroni (port 8008)
    server pg-node1 192.168.1.100:6432 check port 8008 inter 2000 fall 3 rise 2
    server pg-node2 192.168.1.101:6432 check port 8008 inter 2000 fall 3 rise 2
    server pg-node3 192.168.1.102:6432 check port 8008 inter 2000 fall 3 rise 2

```

Health Checks via Patroni API

Patroni provides purpose-built HTTP endpoints for load balancer health checks:

Endpoint	Returns 200	Returns 503
/primary	Node is primary	Not primary
/replica	Node is replica (not lagging)	Not replica or lagging
/replica?lag=<bytes>	Replica with lag < specified	Replica with lag >= specified
/standby-leader	Standby leader	Not standby leader
/read-only	Primary OR replica	Unhealthy

/health	Any healthy state	Unhealthy
/async	Any async replica	-
/asynchronous	Any async replica	-

```
# Test health check endpoints manually
curl -I http://192.168.1.100:8008/primary
# HTTP/1.0 200 OK (if this is the primary)
# HTTP/1.0 503 Service Unavailable (if not primary)

curl -I http://192.168.1.101:8008/replica
# HTTP/1.0 200 OK (if this is a replica)

# Check with lag limit (only replicas with < 10MB lag)
curl -I "http://192.168.1.101:8008/replica?lag=10485760"

# Get full status JSON
curl -s http://192.168.1.100:8008/ | python3 -m json.tool
```

Routing Reads to Replicas

Application-Level Read/Write Splitting

```
# Python example: two connection strings, one for writes, one for reads
import psycopg2

WRITE_DSN = "host=haproxy-vip port=5000 dbname=mydb user=appuser"
READ_DSN = "host=haproxy-vip port=5001 dbname=mydb user=appuser"

def get_write_conn():
    return psycopg2.connect(WRITE_DSN)

def get_read_conn():
    return psycopg2.connect(READ_DSN)

# Write operation
with get_write_conn() as conn:
    with conn.cursor() as cur:
        cur.execute("INSERT INTO orders (user_id, total) VALUES (%s, %s)", (1,
99.99))

# Read operation (goes to replica)
with get_read_conn() as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM products WHERE category = %s", ('electronics',))
        products = cur.fetchall()
```

Framework Examples

```
# Ruby on Rails database.yml
production:
  primary:
    adapter: postgresql
    host: haproxy-vip
    port: 5000    # writes
    database: mydb
    username: appuser
    password: <%= ENV['DB_PASSWORD'] %>

  replica:
    adapter: postgresql
    host: haproxy-vip
    port: 5001    # reads
    database: mydb
    username: appuser
    password: <%= ENV['DB_PASSWORD'] %>
    replica: true
```

HAProxy Stats Dashboard

```
# Enable HAProxy stats dashboard (in haproxy.cfg)
frontend stats
  mode http
  bind *:7000
  stats enable
  stats uri /stats
  stats refresh 10s
  stats auth admin:secure_password
  stats show-legends
  stats show-node
```

```
# Access from command line
# View backend status
echo "show stat" | socat /run/haproxy/admin.sock stdio

# Using haproxy admin socket
echo "show info" | socat /run/haproxy/admin.sock stdio | grep -E
"Maxconn|CurrConns|Uptime"

# Check specific backend
echo "show stat" | socat /run/haproxy/admin.sock stdio | \
  grep "postgres_primary" | cut -d',' -f1,2,18,19

# Disable a backend server for maintenance
echo "disable server postgres_primary/pg-node1" | socat /run/haproxy/admin.sock
```

```
studio

# Re-enable after maintenance
echo "enable server postgres_primary/pg-node1" | socat /run/haproxy/admin.sock studio

# Set server weight
echo "set weight postgres_replicas/pg-node2 150%" | socat /run/haproxy/admin.sock
studio
```

High Availability for HAProxy

HAProxy itself can be a single point of failure. Use Keepalived for VIP failover.

```
HAProxy1 (Active)  ←VIP: 192.168.1.200→ HAProxy2 (Passive)
|
| If HAProxy1 fails, VIP moves to HAProxy2
| Applications still connect to VIP
|
```

```
# Install keepalived on both HAProxy nodes
sudo apt-get install -y keepalived
```

```
# /etc/keepalived/keepalived.conf on HAProxy1 (MASTER)
vrrp_script chk_haproxy {
    script "killall -0 haproxy"    # Check if haproxy is running
    interval 2
    weight 2
}

vrrp_instance VI_1 {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 200                    # Higher priority = preferred master

    authentication {
        auth_type PASS
        auth_pass keepalived_secret
    }

    virtual_ipaddress {
        192.168.1.200/24            # The VIP
    }

    track_script {
        chk_haproxy
    }
}
```



```
# /etc/keepalived/keepalived.conf on HAProxy2 (BACKUP)
# Same, but:
# state BACKUP
# priority 100 (lower than master)
```

```
sudo systemctl enable keepalived
sudo systemctl start keepalived

# Verify VIP is on master
ip addr show eth0 | grep 192.168.1.200
```

Integration Patterns

Pattern 1: HAProxy + PgBouncer + Patroni (Production Standard)

```
Apps —> HAProxy (VIP) —> PgBouncer (per node) —> PostgreSQL (Patroni)
      port 5000/5001      port 6432      port 5432
      health via      multiplexing      HA managed
      Patroni :8008      connections      by Patroni
```

Pattern 2: HAProxy Only (Simple)

```
Apps —> HAProxy (VIP) —> PostgreSQL (Patroni)
      port 5000/5001      port 5432
      (no pooling)
```

Pattern 3: Consul + HAProxy with Service Discovery

```
# Consul-template watches Patroni/Consul for leader changes
# Automatically regenerates haproxy.cfg and reloads
consul-template \
  -template "/etc/haproxy/haproxy.ctmpl:/etc/haproxy/haproxy.cfg:service haproxy
reload"
```

Common Mistakes

1. **Mode http instead of mode tcp** — PostgreSQL is a binary protocol, not HTTP
2. **Health check port 5432 instead of 8008** — checking PostgreSQL connectivity, not primary status; all nodes pass
3. **No fall/rise tuning** — too fast: flapping; too slow: long failover detection
4. **Single HAProxy node** — HAProxy becomes the single point of failure it's meant to eliminate
5. **Not testing health check endpoints** — `curl` the endpoints manually before deploying
6. **Client/server timeout too short** — long-running queries get terminated
7. **Not logging** — harder to debug connectivity issues
8. **Same weight for all backends without intent** — uneven load distribution

9. **Not disabling server for maintenance** — draining connections gracefully before taking a node offline

Best Practices

1. Use **Patroni REST API endpoints** for health checks (not TCP connect)
2. Set **inter 2000 fall 3 rise 2** for 6-second failure detection
3. Run **two HAProxy instances** with Keepalived VIP
4. Use **maxconn per server** to prevent overloading backends
5. **Monitor HAProxy stats** (use stats dashboard + alerting)
6. Set **appropriate timeouts** — 30 minutes for `client` and `server` for long queries
7. Use **weight** to send more reads to replicas: `weight 3` for replicas vs primary
8. **Test failover** through HAProxy: verify connections auto-recover

Performance Considerations

```
# High-throughput tuning
global
    maxconn 50000          # Increase global connection limit
    nbthread 4             # Multi-threaded mode (HAProxy 1.8+)
    cpu-map auto:1/1-4 0-3 # Pin threads to CPUs

defaults
    timeout tunnel 3600s    # Long-lived tunnels (for streaming replicas)

backend postgres_primary
    timeout connect 1s      # Fast failure detection
    option tcp-smart-connect # TCP fast open
```

Interview Questions

Q1: Why use `mode tcp` instead of `mode http` for PostgreSQL in HAProxy?

A: PostgreSQL uses its own binary wire protocol, not HTTP. `mode tcp` passes bytes transparently without attempting HTTP parsing. `mode http` would corrupt the PostgreSQL protocol. The exception is health checks, which use HTTP to talk to Patroni's REST API — but the actual database traffic uses TCP mode.

Q2: How does HAProxy know which PostgreSQL node is the primary?

A: Via HTTP health checks against Patroni's REST API (port 8008). HAProxy sends `GET /primary` to each backend's port 8008. The node that is currently the Patroni leader returns HTTP 200; all others return 503. HAProxy marks nodes returning 503 as DOWN for the primary backend.

Q3: What is the purpose of `fall 3 rise 2` in HAProxy server configuration?

A: `fall 3` means a server is marked DOWN after 3 consecutive failed health checks. `rise 2` means it's marked UP after 2 consecutive successful checks. Combined with `inter 2000` (check every 2 seconds): failure detection takes ~6 seconds, recovery takes ~4 seconds. This prevents flapping due to transient health check failures.

Q4: How do you prevent HAProxy itself from being a single point of failure?

A: Deploy two HAProxy instances with Keepalived managing a virtual IP (VIP). HAProxy1 holds the VIP in MASTER state. If HAProxy1 fails (or haproxy process stops), Keepalived detects it and moves the VIP to HAProxy2 (BACKUP). Applications connect to the VIP and experience only brief (sub-second) interruption.

Q5: What is the difference between the `/replica` and `/read-only` Patroni endpoints?

A: `/replica` returns 200 only for healthy replicas (not the primary). `/read-only` returns 200 for both replicas AND the primary. Use `/replica` when you want reads to go ONLY to replicas. Use `/read-only` when you want reads distributed across all nodes including primary (primary falls back when no replicas are available).

Q6: How do you gracefully drain a PostgreSQL backend in HAProxy for maintenance?

A: Use the admin socket: `echo "disable server postgres_primary/pg-node1" | socat /run/haproxy/admin.sock stdio`. This stops new connections from being assigned to that server while existing connections finish naturally. After maintenance, re-enable: `echo "enable server postgres_primary/pg-node1" | socat ...`

Q7: How does HAProxy handle a Patroni failover?

A: When the primary fails, Patroni promotes a replica. The new primary's Patroni API starts returning 200 for `/primary`; the old primary returns 503. HAProxy detects this within `fall * inter` seconds (e.g., 6 seconds) and starts routing traffic to the new primary automatically. No manual HAProxy reconfiguration is needed.

Q8: What does the HAProxy `balance roundrobin` directive do for the replica backend?

A: It distributes connections across all UP servers in the backend in round-robin order. For the replica backend, all healthy replicas are included, so read queries are distributed evenly. The primary is also listed but fails the `/replica` health check so it's excluded (unless using `/read-only`).

Exercises and Solutions

Exercise 1: Test Health Check Behavior

```
#!/bin/bash
# Test Patroni health check endpoints from HAProxy perspective

for NODE in 192.168.1.100 192.168.1.101 192.168.1.102; do
    echo "=== Node: $NODE ==="

    HTTP_CODE=$(curl -o /dev/null -s -w "%{http_code}" http://${NODE}:8008/primary)
    echo "  /primary: $HTTP_CODE $([ $HTTP_CODE -eq 200 ] && echo 'IS PRIMARY' || echo 'not primary')"

    HTTP_CODE=$(curl -o /dev/null -s -w "%{http_code}" http://${NODE}:8008/replica)
    echo "  /replica: $HTTP_CODE $([ $HTTP_CODE -eq 200 ] && echo 'IS REPLICIA' || echo 'not replica')"

    HTTP_CODE=$(curl -o /dev/null -s -w "%{http_code}" http://${NODE}:8008/health)
    echo "  /health: $HTTP_CODE $([ $HTTP_CODE -eq 200 ] && echo 'HEALTHY' || echo 'NOT HEALTHY')"
```

```
'UNHEALTHY' )"
done
```

Exercise 2: Failover Test Through HAProxy

```
#!/bin/bash
# Continuous connection test during failover

PRIMARY_PORT=5000
HAPROXY_HOST=192.168.1.200

echo "Starting continuous writes to $HAPROXY_HOST:$PRIMARY_PORT"
echo "Simulate failover by stopping Patroni on primary node"

i=0
while true; do
    START=$(date +%s%N)

    RESULT=$(psql "host=$HAPROXY_HOST port=$PRIMARY_PORT dbname=testdb
user=testuser" \
        -c "INSERT INTO failover_test (id, ts) VALUES ($i, now()) RETURNING id;"
2>&1)

    END=$(date +%s%N)
    DURATION=$(( (END - START) / 1000000 ))

    if echo "$RESULT" | grep -q "$i"; then
        echo "$(date): INSERT $i OK (${DURATION}ms)"
    else
        echo "$(date): INSERT $i FAILED: $RESULT"
    fi

    i=$((i + 1))
    sleep 1
done
```

Cross-References

- [06_patroni.md](#) — Patroni REST API details
- [07_pgouncer.md](#) — PgBouncer configuration
- [09_ha_architecture_patterns.md](#) — Complete HA architectures